

Relatório – Aula 07

Introdução

Pelo que entendi dos slides, o ponto central da aula é a diferença entre o que o código parece fazer e o que a linguagem realmente executa. Muitos erros não vêm da regra de negócio, e sim de detalhes como precedência de operadores, conversão implícita de tipos, ordem de avaliação, um if sem chaves ou um laço que não avança. Abaixo explico os principais conceitos com os exemplos da aula, incluindo três casos reais: o type juggling do PHP, o travamento dos Zune e o “goto fail” da Apple.

1. Precedência e associatividade

A **precedência** diz qual operador é resolvido primeiro e a **associatividade** diz como operadores de mesma precedência se agrupam. No exemplo em Python, $a + b * c$ dá 23 porque o $*$ vem antes do $+$, e só os parênteses mudam isso. O que mais me chamou atenção foi que $-2 ** 2$ dá -4, porque o $**$ tem precedência maior que o menos unário, e que o $**$ associa à direita ($2 ** 3 ** 2$ é $2^9 = 512$). Também vi que a divisão muda de linguagem: Python arredonda para baixo ($-7 // 2$ é -4), enquanto Java e C truncam em direção a zero.

Código 1) precedencia.py (Python)

```
a, b, c = 3, 4, 5
print("a + b * c =", a + b * c)
print("(a + b) * c =", (a + b) * c)

print("-2 ** 2 =", -2 ** 2)
print("(-2) ** 2 =", (-2) ** 2)

print("2 ** 3 ** 2 =", 2 ** 3 ** 2)
print("(2 ** 3) ** 2 =", (2 ** 3) ** 2)

print("10 - 4 - 3 =", 10 - 4 - 3)

print("-7 // 2 =", -7 // 2, " -7 % 2 =", -7 % 2)
```

Saída:

```
a + b * c = 23
(a + b) * c = 35
-2 ** 2 = -4
(-2) ** 2 = 4
2 ** 3 ** 2 = 512
(2 ** 3) ** 2 = 64
10 - 4 - 3 = 3
-7 // 2 = -4  -7 % 2 = 1
```

2. Efeitos colaterais e ordem de avaliação

Efeito colateral é quando a expressão, além de dar um valor, altera algo no programa. Em $a = a + \text{fun1}()$, a função muda a para 17 e retorna 3. Se o a for lido antes da chamada, o resultado é 8; se a chamada vier primeiro, é 20. Em Java a ordem é garantida (esquerda para a direita), então sempre dá 8. Em C o padrão não define a ordem e o gcc deu 20; em

Go o resultado também foi 20, porque a especificação não diz quando a variável é lida. Concluí que não se deve escrever expressões que dependem da ordem de efeitos colaterais.

Código 2) Ordem.java (Java)

```
public class Main {
    static int a = 5;

    static int fun1() {
        a = 17;
        return 3;
    }

    public static void main(String[] args) {
        a = a + fun1();
        System.out.println("a = " + a);

        int i = 1;
        int j = i++ + i++;
        System.out.println("j = " + j + ", i = " + i);
    }
}
```

Saída:

```
a = 8
j = 3, i = 3
```

Código 3 – ordem.c (C)

```
#include <stdio.h>

int a = 5;

int fun1(void) {
    a = 17;
    return 3;
}

int main(void) {
    a = a + fun1();
    printf("a = %d\n", a);

    return 0;
}
```

Saída:

```
a = 20
```

3. Sobrecarga de operadores

Sobrecarga é o mesmo operador com significados diferentes conforme os tipos. Em Java o + soma números e concatena textos, então 'a' + 1 dá 98 (o char vira int), mas "" + 'a' + 1 dá a1. Java não deixa o programador sobrecarregar operadores, por isso o BigDecimal

usa métodos como add. Já Python e Rust permitem definir operadores para tipos próprios.

Código 4) Sobrecarga.java (Java)

```
import java.math.BigDecimal;

public class Main {
    public static void main(String[] args) {
        System.out.println(1 + 2);
        System.out.println("1" + 2);
        System.out.println('a' + 1);
        System.out.println("" + 'a' + 1);

        BigDecimal x = new BigDecimal("0.1"), y = new BigDecimal("0.2");
        System.out.println(x.add(y).multiply(new BigDecimal("3")));
    }
}
```

Saída:

```
3
12
98
a1
0.9
```

4. Conversões de tipos e coerção

No **alargamento** o valor vai para um tipo maior, mas ainda pode perder precisão: em Java, 123456789 virou 123456792 ao passar por float. No **estreitamento** é preciso um cast e há perda (3.99 vira 3 e (byte) 200 vira -56). Também vi que $1 / 2$ é 0, mas $1 / 2.0$ é 0.5, porque o 1 sofre coerção para double. Go e Rust não fazem coerção implícita e exigem conversão explícita.

Código 5 – Conversoes.java (Java)

```
public class Main {
    public static void main(String[] args) {
        int grande = 123456789;
        float f = grande;
        System.out.println(f);
        System.out.println((int) f);

        double d = 3.99;
        System.out.println((int) d);
        System.out.println((byte) 200);

        byte b = 10, c = 20;
        byte soma = (byte) (b + c);
        System.out.println(soma);

        System.out.println(1 / 2);
    }
}
```

```

        System.out.println(1 / 2.0);
    }
}

```

Saída:

```

1.2345679E8
123456792
3
-56
30
0
0.5

```

O JavaScript mostra bem a **coerção implícita**: "5" - 2 dá 3, mas "5" + 2 dá "52". O == converte os tipos antes de comparar (0 == "" e 0 == "0" são verdadeiros, mas "" == "0" é falso), e o === não converte, por isso é a regra prática recomendada.

Código 6) coercao.js (JavaScript)

```

console.log("5" - 2);
console.log("5" + 2);
console.log("5" * "2");
console.log(true + 1);
console.log([] + []);
console.log([] + {});
console.log(null + 1, undefined + 1);
console.log("b" + "a" + +"a" + "a");

console.log(0 == "", 0 == "0", "" == "0");
console.log(0 === "", 0 === "0");
console.log(Number("12px"), parseInt("12px"));

```

Saída:

```

3
52
10
2

object Object
1 NaN
baNaN
true true false
false false
NaN 12

```

Caso real: PHP (type juggling). Com ==, textos que parecem números são convertidos: "0e123" e "0e456" são lidos como notação científica, os dois valem zero e a comparação dá verdadeiro. Os hashes md5("240610708") e md5("QNKCDZO") começam com 0e seguido de dígitos, então um login que compara hashes com == aceitaria a senha errada. Aqui uma conversão que parece comodidade virou uma falha de segurança.

Código 7) coercao.php (PHP)

```

<?php

```

```

var_dump("0e123" == "0e456");
var_dump("0e123" === "0e456");
var_dump("1" == "01");
var_dump(100 == "1e2");
var_dump("abc" == 0);

var_dump(md5("240610708"));
var_dump(md5("QNKCDZO"));
var_dump(md5("240610708") == md5("QNKCDZO"));

```

Saída:

```

bool(true)
bool(false)
bool(true)
bool(true)
bool(false)
string(32) "0e462097431906509019562988736854"
string(32) "0e830400451993494058024219903391"
bool(true)

```

5. Expressões booleanas e curto-circuito

Em C, $a > b > c$ com 3, 2 e 1 dá 0, pois $3 > 2$ vale 1 e depois $1 > 1$ é falso. Para expressar a ideia é preciso $(a > b) \&\& (b > c)$. No mesmo exemplo, `if (x = 0)` é uma atribuição e não uma comparação: o `x` é sobrescrito. Python é diferente e encadeia as comparações, então $a > b > c$ dá `True`.

Código 8) relacionais.c (C)

```

#include <stdio.h>

int main(void) {
    int a = 3, b = 2, c = 1;
    printf("%d\n", a > b > c);
    printf("%d\n", (a > b) && (b > c));

    int x = 5;
    if (x = 0)
        printf("x é zero\n");
    else
        printf("x = %d (foi sobrescrito!)\n", x);

    if (-1)
        printf("-1 conta como verdadeiro em C\n");
    return 0;
}

```

Saída:

```

0
1
x = 0 (foi sobrescrito!)
-1 conta como verdadeiro em C

```

No **curto-circuito**, o segundo operando só é avaliado se for necessário. Em Java, `indice < lista.length && lista[indice] != chave` evita acessar fora do vetor. Com `&&` o método `caro()` não é chamado, mas com `&` os dois lados são avaliados, e por isso `s != null & s.length() > 0` gera `NullPointerException`.

Código 9) CurtoCircuito.java (Java)

```
public class Main {
    static int chamadas = 0;

    static boolean caro() {
        chamadas++;
        return true;
    }

    public static void main(String[] args) {
        int[] lista = {4, 8, 15};
        int chave = 99, indice = 0;
        while (indice < lista.length && lista[indice] != chave)
            indice++;
        System.out.println("indice = " + indice);

        boolean r1 = false && caro();
        boolean r2 = false & caro();
        System.out.println("chamadas = " + chamadas);

        String s = null;
        System.out.println(s != null && s.length() > 0);
        try {
            System.out.println(s != null & s.length() > 0);
        } catch (NullPointerException e) {
            System.out.println("com & : NullPointerException");
        }
    }
}
```

Saída:

```
indice = 3
chamadas = 1
false
com & : NullPointerException
```

6. Atribuição

Em C a atribuição é uma expressão e devolve valor, o que permite `sum = count = 0` ou `while ((ch = texto[i++]) != '\0')`. Em Python e Go ela não é expressão, então `if x = 5` não compila (Python tem o operador `:=` como exceção). Em Rust as variáveis são imutáveis por padrão e é preciso usar `mut`. Na atribuição de modo misto, C converte em silêncio: `int i = 3.7` vira 3, `char c = 300` vira 44 e `unsigned u = -1` vira 4294967295. Java exige `cast`, mas o `+=` tem `cast` embutido, então `i += 3.7` compila.

Código 10) modo_misto.c (C)

```
#include <stdio.h>

int main(void) {
    int i = 3.7;
    char c = 300;
    unsigned int u = -1;
    printf("%d %d %u\n", i, c, u);
    return 0;
}
```

Saída:

3 44 4294967295

7. Seleção

No **else pendente**, o else pertence ao if mais próximo que ainda não tem else, mesmo que a indentação sugira outra coisa. No exemplo, sem chaves o resultado é 1 (o else é do if interno); com chaves, o else passa a ser do if externo e o resultado é -1.

Código 11) else_pendente.c (C)

```
#include <stdio.h>

int main(void) {
    int sum = 0, count = 5, result = -1;
    if (sum == 0)
        if (count == 0)
            result = 0;
    else
        result = 1;
    printf("result = %d\n", result);

    result = -1;
    if (sum == 0) {
        if (count == 0)
            result = 0;
    } else
        result = 1;
    printf("result = %d\n", result);
    return 0;
}
```

Saída:

result = 1
result = -1

No switch de C, sem break a execução continua nos próximos case (fall-through): com index = 1, os dois contadores foram incrementados e ainda caiu no default. Go e o switch com -> do Java evitam esse problema, e o match de Rust obriga a cobrir todos os casos.

Código 12) switch.c (C)

```

#include <stdio.h>

void sem_break(int index) {
    int odd = 0, even = 0;
    switch (index) {
        case 1:
        case 3: odd += 1;
        case 2:
        case 4: even += 1;
        default: printf(" Error in switch, index = %d\n", index);
    }
    printf("sem break: index=%d odd=%d even=%d\n", index, odd, even);
}

void com_break(int index) {
    int odd = 0, even = 0;
    switch (index) {
        case 1:
        case 3: odd += 1; break;
        case 2:
        case 4: even += 1; break;
        default: printf(" Error in switch, index = %d\n", index);
    }
    printf("com break: index=%d odd=%d even=%d\n", index, odd, even);
}

int main(void) {
    sem_break(1);
    com_break(1);
    return 0;
}

```

Saída:

```

Error in switch, index = 1
sem break: index=1 odd=1 even=1
com break: index=1 odd=1 even=0

```

8. Iteração

Um laço precisa de uma condição de parada e de progresso em direção a ela. Em JavaScript, `for...in` percorre as chaves e `for` percorre os valores, por isso a soma com `for` deu "0012" e não 60. Python e Rust também têm iteração por valores e Rust ainda tem `break` rotulado.

Código 13) `iteracao.js` (JavaScript)

```

const notas = [10, 20, 30];

let total = 0;
for (const i in notas) total += i;
console.log(total);

total = 0;

```

```

for (const v of notas) total += v;
console.log(total);

notas.forEach((v, i) => console.log(i, v));

function* contagem(n) {
  for (let i = n; i > 0; i--) yield i;
}
console.log([...contagem(3)]);

```

Saída:

```

0012
60
0 10
1 20
2 30
[ 3, 2, 1 ]

```

Caso real: Zune. Em 31/12/2008 os Zune de 30 GB travaram. O laço while (dias > 365) convertia dias em ano. Em ano bissexto, quando dias vale 366, a condição dias > 366 é falsa: dias não diminui e ano não aumenta, então a próxima volta é igual à anterior e o laço nunca termina. O erro só aparecia nessa combinação (ano bissexto e dia 366). Aprendi que é preciso verificar se **cada caminho** do laço altera as variáveis da condição. Na versão simplificada, um contador interrompe a demonstração.

Código 14) zune.c (C)

```

#include <stdio.h>

int bissexto(int ano) {
  return (ano % 4 == 0 && ano % 100 != 0) || ano % 400 == 0;
}

int main(void) {
  int dias = 366;
  int ano = 2008;
  long voltas = 0;

  while (dias > 365) {
    if (bissexto(ano)) {
      if (dias > 366) {
        dias -= 366;
        ano += 1;
      }
    } else {
      dias -= 365;
      ano += 1;
    }
    if (++voltas == 1000000) {
      printf("laço infinito! dias=%d ano=%d depois de %ld voltas\n", dias, ano, voltas);
      return 1;
    }
  }
}

```

```

    printf("ano = %d\n", ano);
    return 0;
}

```

Saída:

laço infinito! dias=366 ano=2008 depois de 1000000 voltas

9. Desvio incondicional: o ‘goto fail’ da Apple

No código simplificado, a segunda linha goto fail; parece pertencer ao if, mas como não há chaves só a primeira instrução pertence a ele. A segunda é executada sempre e pula direto para fail, antes de verifica_assinatura(). Como err ainda vale 0, a função devolve “certificado válido” mesmo com assinatura falsa. A indentação não tem significado para o compilador; chaves obrigatórias, como em Go e Rust, evitariam o erro. Na aula também vi os comandos protegidos de Dijkstra, em que a escolha entre guardas verdadeiras é aleatória (select de Go), e as guardas de Haskell, que são testadas em ordem.

Código 15) goto_fail.c (C)

```

#include <stdio.h>

int verifica_hash(void)    { return 0; }
int verifica_assinatura(void) { return -1; }

int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail;
    if ((err = verifica_assinatura()) != 0)
        goto fail;
fail:
    return err;
}

int main(void) {
    printf("resultado = %d (0 significa certificado válido)\n", valida_certificado());
    return 0;
}

```

Saída:

resultado = 0 (0 significa certificado válido)

10. Exercícios

Nos exercícios “encontre o bug”, o erro está em um detalhe da linguagem e não na lógica. Segue o que entendi de cada um.

Questão 1. O for percorre as chaves ("0", "1", "2"), então total += p concatena texto em vez de somar. O correto é usar for...of.

Código 16q1.js (JavaScript)

```
const precos = [10, 20, 30];
```

```
let total = 0;
for (const p in precos) total += p;
console.log("Total: " + total);
```

Saída:

Total: 0012

Questão 2. O `d or 10` trata o 0 como falso e o troca por 10, então o desconto de 0% vira 10% e o resultado é 180.0 em vez de 200. O correto é testar `d is None`.

Código 17) q2.py (Python)

```
def desconto(preco, d=None):
    d = d or 10
    return preco * (100 - d) / 100
```

```
print(desconto(200, 0))
```

Saída:

180.0

Questão 3. Sem `break`, a execução cai do case 2 até o default, e o último valor atribuído é "inválido".

Código 18) Q3.java (Java)

```
public class Main {
    public static void main(String[] args) {
        int dia = 2;
        String nome = "";
        switch (dia) {
            case 1: nome = "domingo";
            case 2: nome = "segunda";
            case 3: nome = "terça";
            default: nome = "inválido";
        }
        System.out.println(nome);
    }
}
```

Saída:

inválido

Questão 4. O `;` depois do `if` faz o `if` controlar um comando vazio, então o `printf` seguinte é executado sempre, mesmo com a condição falsa.

Código 19) q4.c (C)

```
#include <stdio.h>
```

```
int main(void) {
    int saldo = 100, saque = 50;
    if (saque > saldo);
    printf("saldo insuficiente\n");
}
```

```
    printf("fim\n");
    return 0;
}
```

Saída:

```
saldo insuficiente
fim
```

Questão 5. Dentro do if, $x := x * 2$ declara uma nova variável no bloco, e o x de fora não muda. Para alterar o valor externo seria $x = x * 2$.

Código 20) q5.go (Go)

```
package main

import "fmt"

func main() {
    x := 10
    if x > 5 {
        x := x * 2
        fmt.Println("dentro:", x)
    }
    fmt.Println("fora:", x)
}
```

Saída:

```
dentro: 20
fora: 10
```

Questão 6. acertos / total é divisão inteira ($7 / 10 = 0$), então o resultado é 0.0%. O correto seria $\text{acertos} * 100.0 / \text{total}$, que dá 70.0%.

Código 21)Q6.java (Java)

```
public class Main {
    public static void main(String[] args) {
        int acertos = 7, total = 10;
        double taxa = acertos / total * 100;
        System.out.println(taxa + "%");
    }
}
```

Saída:

```
0.0%
```

11. Autopesquisa: expressões em Ruby e em Lisp

Em Ruby os operadores são métodos: $1 + 2$ é o mesmo que $1.+(2)$, uma chamada de método do objeto 1. O $**$ associa à direita (512) e os laços usam iteradores com blocos, como $3.\text{times}$ e each_with_index .

Código 22) expressoes.rb (Ruby)

```
puts 1 + 2 # 3
puts 1.+(2) # 3: o + é uma chamada de método do objeto 1
puts 2 ** 3 ** 2 # 512: ** associa à direita
3.times { |i| print i, " " } # iterador com bloco: 0 1 2
```

```
puts  
[10, 20, 30].each_with_index { |v, i| puts "#{i}: #{v}" }  
puts (1..5).select(&:even?).inspect # [2, 4]
```

Saída:

```
3  
3  
512  
0 1 2  
0: 10  
1: 20  
2: 30  
[2, 4]
```

Em Lisp a notação é prefixa e toda entre parênteses, então **não existe precedência nem associatividade**: (+ a (* b c)) e (* (+ a b) c) já mostram a ordem. Um mesmo operador aceita vários operandos, como (+ 1 2 3 4 5), e o if é uma expressão que devolve valor.

Código 23) expressoes.lisp (Common Lisp)

```
(defvar a 3)  
(defvar b 4)  
(defvar c 5)  
(format t "a + b * c = ~a~%" (+ a (* b c))) ; 23: os parênteses dizem a ordem  
(format t "(a + b) * c = ~a~%" (* (+ a b) c)) ; 35  
(format t "soma de 5 = ~a~%" (+ 1 2 3 4 5)) ; um operador, vários operandos  
(format t "if = ~a~%" (if (> a 0) a (* 2 a))) ; expressão condicional
```

Saída:

```
a + b * c = 23  
(a + b) * c = 35  
soma de 5 = 15  
if = 3
```